MSFLib Integration & Developer Experience Report

Gridnova AI — Intelligent Energy Monitoring and Load Management

Prepared for: Hackathon organizers, MSFLib maintainers, and project reviewers
Project team: Team GridNova
Evaluation date: 16 October 2026
Evidence basis: Local integration logs, TypeScript declarations, README excerpts, build outputs, and dashboard tests shared during development.

Executive summary

Gridnova AI integrated genuine @msflib/react-components widgets into a React 19 / TypeScript / Vite dashboard, including an operational FormBuilder for remote simulated power-threshold configuration, an events table, and a loading skeleton. We verified successful TypeScript production builds and an end-to-end threshold update from the interface to the ESP32-reported state (85 W in the observed test). The library enabled rapid assembly of reusable interfaces, but integration required direct inspection of installed declaration files and package documentation. Styling, form configuration, bundle size, and authentication/backend compatibility were the main friction points.

1. Project and integration context

Gridnova AI combines an ESP32 edge device, relay-controlled demonstration loads, OLED display, DS18B20 temperature monitoring, PZEM electrical measurements where valid, a FastAPI service deployed on Render, and a React dashboard. The dashboard provides Overview, Live Monitoring, Load Control, Faults & Events, Reports & Analytics, and Settings views. Device commands are queued through the backend and verified against subsequent device telemetry rather than treated as immediately applied.

2. MSFLib packages installed

Package	Version	Observed role
@msflib/core	0.0.10	Installed; application configuration interfaces explored
@msflib/typescript	0.0.10	Installed; shared type support
@msflib/react-components	0.0.46	FormBuilder, events table, loading skeleton used in dashboard
@msflib/react-auth	0.0.22	API and README inspected; not used for live

		authentication
@msflib/react-workspace	0.0.14	Installed; no confirmed functional integration in supplied evidence
@msflib/react-ai	0.0.5	Installed; no confirmed functional integration in supplied evidence

Package installation alone is not evidence that every module was used. This report distinguishes installed dependencies from functionality verified in the running application.

3. Features implemented and validated

MSFLib FormBuilder — load threshold

A configurable numeric input and submit action were rendered for a simulated demand threshold of 1–999 W. Initial attempts produced a TypeScript TS2741 error because the submit FormElement lacked a required name property. After correcting the element structure and styling, the form appeared and a submitted 85 W threshold was subsequently shown as the current ESP32 threshold in the dashboard. This is stronger evidence than a button click alone, though the exact internal device acknowledgement path was not independently audited.

MSFLib events table

The Fault & Event History view displayed timestamped operational events, severity, device name, search input, pagination, and selection checkboxes. Observed events included automatic load shedding, estimated demand limit exceeded, and demand-limit condition cleared.

MSFLib AppSkeleton

The package type declarations confirmed AppSkeleton and nested Avatar, Text, Image, and Button components. A loading component was integrated into the dashboard and the final production build succeeded after initial unused-import/state errors were resolved.

Production build

Vite 8.3.4 and TypeScript successfully built the application, transforming approximately 1,680 modules. The main JavaScript output was approximately 1.3 MB minified and 392 KB gzip, with a non-fatal chunk-size warning.

4. Authentication assessment

Inspection of @msflib/react-auth v0.0.22 showed AuthProvider, useAuth, createAuthApi, and typed login/logout functions. Its documented LoginPayload requires email and password, while AuthUser expects access_token, token_type, and expires; the quick-start guide uses TanStack Query and configurable authentication endpoints. Gridnova AI's working Express gateway instead uses a password-only login, server-side sessions, and HttpOnly cookies. Consequently, the application retained its own authentication implementation rather than claiming MSFLib authentication

integration. Local login and logout were tested successfully. The production gateway keeps the backend admin key on the server, not in browser code.

5. Issues encountered and lessons learned

Observation	Evidence / impact	Suggested improvement
TypeScript contract discovery	The submit element initially omitted the required name field (TS2741). Inspecting formbuilder.d.ts clarified the expected shape.	Provide copy-ready, versioned TypeScript examples for all form element types.
Form rendering and styling	The form initially rendered incompletely or with poorly contrasted input/label styling; the team inspected package files and iterated on configuration and CSS.	Ship a dark-theme preset and a full FormBuilder example with input, validation, and submit button.
Missing expected distribution filename	An attempted search for formbuilder.js failed because the package distributed formbuilder.cjs and formbuilder.mjs.	Document ESM/CJS output filenames and debugging guidance.
Integration warnings and build discipline	Unused loading imports/state caused TS6133 errors until the loading component was actually used. The production bundle emitted a large-chunk warning.	Include an end-to-end Vite integration example and guidance on tree-shaking/lazy loading.
Authentication mismatch	MSFLib auth expects token-based responses and an email/password contract, whereas the gateway uses cookie sessions and password-only login.	Support configurable adapters for cookie-based sessions and flexible login payload/response shapes.
Dependency friction	Installing Express and rate limiting initially produced an ERESOLVE dependency error; subsequent installation and npm ls checks showed Express 5.3.0 and express-rate-limit 8.7.1 installed. This was a project dependency issue, not demonstrated to be an MSFLib defect.	Provide tested reference dependency matrices for supported React/Vite/Node versions.

6. Developer experience assessment

Strengths: reusable component APIs; typed exports that enabled troubleshooting; an operational form and table without building every UI primitive from scratch; React 19 compatibility in the tested project; documentation for auth providers and configuration. Challenges: limited immediately discoverable copy-and-paste examples for the precise package version, need to inspect declarations and compiled distribution files, default visual styles requiring dark-theme adaptation, and auth contract assumptions not matching a custom IoT gateway.

Overall experience: productive after initial setup, but not fully plug-and-play. The most successful workflow was to confirm the installed version, inspect its .d.ts definitions, implement one component at a time, test behavior in the browser, and run a strict TypeScript production build.

7. Prioritized recommendations to MSFLib maintainers

- P1 — Publish version-matched FormBuilder examples for text, number, submit, validation, loading, and async submission.
- P1 — Add documented auth adapters for server sessions/HttpOnly cookies, non-email identifiers, and custom response formats.
- P1 — Provide first-class dark-mode tokens or themes and accessibility-tested default contrast.
- P2 — Publish a React + Vite + TypeScript starter integrating FormBuilder, table, skeleton, and authentication.
- P2 — Provide a concise compatibility matrix for React, TypeScript, Node, TanStack Query, and peer dependencies.
- P2 — Offer tree-shaking guidance and lazy-loading patterns to reduce production JavaScript bundles.
- P3 — Include component playgrounds, troubleshooting notes, and changelogs highlighting type-contract changes.

8. Recommendations for the Gridnova AI team

- Keep the Express session gateway and admin-key proxy for public deployment; do not expose secrets in the Vite client or repository.
- Document the exact MSFLib components used, their versions, and screenshots of successful operation.
- Retain separate labels for measured PZEM values and estimated/simulated demand; do not infer certified electrical protection.
- Use persistent storage for reports if historical accuracy beyond in-memory uptime is required.
- Complete end-to-end production tests for login, logout, telemetry, command acknowledgement, and report export after deployment.

9. Limitations and evidence notes

This is a project-specific developer experience report, not a formal benchmark, security audit, or exhaustive MSFLib feature review. Some packages were installed but not functionally verified. The backend's Python MSFLib dependency was previously inaccessible during deployment, so the live

backend uses a standalone FastAPI implementation. The MSFLib auth package was evaluated from its README and types, not deployed as the project's authentication provider. Build size reflects the entire application and cannot be attributed solely to MSFLib. Recommendations are based on observed friction and are not claims that maintainers have already accepted or addressed them.

10. Conclusion

MSFLib made a practical contribution to Gridnova AI's interface development, particularly threshold configuration, event presentation, and loading states. The experience supports using the library for composable React interfaces while highlighting opportunities to improve documentation, styling, compatibility, and authentication flexibility. The project's most defensible outcome is a verified, working MSFLib-backed dashboard feature connected to a live ESP32 workflow, with clearly stated boundaries around the modules that were only inspected or installed.